Sub-Agent-System

Table of Contents

- Chapter 1: Project Overview and File Map
- Chapter 2: Type System and Data Structures
- Chapter 3: Agent Creation and Startup Flow
- Chapter 4: Message Loop and Query Engine
- Chapter 5: Tool Filtering and Permission System
- Chapter 6: Fork Subagent Algorithm
- Chapter 7: Built-in Agent Details
- Chapter 8: Persistent Memory System
- Chapter 9: Task State Machine and Lifecycle
- Chapter 10: Isolation Mechanisms
- Chapter 11: Coordinator Multi-Agent Orchestration
- Chapter 12: End-to-End Data Flow
- Chapter 13: Key Algorithm Pseudocode Implementations
- Chapter 14: Design Patterns and Architectural Philosophy
- Chapter 15: Security and Boundary Control
- Appendix: Constants Quick Reference

Chapter 1: Project Overview and File Map

1.1 File Responsibility Quick Reference

File	Lines	Core Responsibility	Key Exports
`AgentTool.tsx`	1398	Entry: argument parsing, routing logic, result rendering	`AgentTool.call()`, `AgentToolComponent`
`runAgent.ts`	974	Execution loop: context init, query iteration, cleanup	`runAgent()`
`agentToolUtils.ts`	687	Tool filtering, result aggregation, async lifecycle	`filterToolsForAgent()`, `runAsyncAgentLifecycle()`
`loadAgentsDir.ts`	756	Agent definition parsing: md/json/plugins	`getAgentDefinitionsWithOverrides()`
`forkSubagent.ts`	211	Fork logic: context inheritance, message cloning	`buildForkedMessages()`
`prompt.ts`	288	System prompt construction	`getPrompt()`
`LocalAgentTask.tsx`	683	Task state machine, progress tracking, notification enqueuing	`registerAsyncAgent()`, `enqueueAgentNotification()`
`forkedAgent.ts`	690	Fork-specific context creation, usage tracking	`runForkedAgent()`, `createSubagentContext()`
`coordinatorMode.ts`	370	Coordinator system prompts, worker tool context	`getCoordinatorSystemPrompt()`

File	Lines	Core Responsibility	Key Exports
`agentMemory.ts`	178	Memory directory management, safe path validation	`loadAgentMemoryPrompt()`

1.2 Call Hierarchy

```
User Input
  └─ AgentTool.call()              [AgentTool.tsx:1398]
       ├─ resolveAgentType()       → determines agent type
       ├─ validateMcpRequirements()→ checks MCP dependencies
       ├─ setupIsolation()         → Worktree / Remote
       ├─ determineSyncAsync()     → sync or async
       │
       ├─ [Async Branch]
       │    ├─ registerAsyncAgent()        [LocalAgentTask.tsx]
       │    └─ runAsyncAgentLifecycle()    [agentToolUtils.ts]
       │         └─ runAgent() → query()
       │
       └─ [Sync Branch]
            ├─ registerAgentForeground()  [LocalAgentTask.tsx]
            └─ runAgent()                 [runAgent.ts]
                 ├─ createSubagentContext()   [forkedAgent.ts]
                 ├─ initializeAgentMcpServers()
                 ├─ preloadSkills()
                 └─ for await message of query()
                      └─ yield messages → parent session
```

Chapter 2: Type System and Data Structures

2.1 AgentTool Call Input

```python
// AgentTool.tsx: Input Zod Schema
const AgentToolInput = z.object({
  description:  z.string().describe("Agent task description"),
  prompt:       z.string().describe("Instructions for the agent"),
  subagent_type: z.string().optional()
                  .describe("Specialized agent type: Explore | Plan | general-purpose"),
  model:        z.enum(["sonnet", "opus", "haiku"]).optional()
                  .describe("Model override"),
  run_in_background: z.boolean().optional()
                  .describe("Run in background (async)"),
  name:         z.string().optional()
                  .describe("Agent name for SendMessage routing"),
  team_name:    z.string().optional()
                  .describe("Agent team for multi-agent spawning"),
  mode:
z.enum(["bubble","plan","acceptEdits","bypassPermissions","auto"]).optional(),
  isolation:     z.enum(["worktree","remote"]).optional()
                  .describe("Execution isolation mode"),
  cwd:          z.string().optional()
```

```
                    .describe("Override working directory"),
});
```

2.2 Complete Agent Definition Types

```
// loadAgentsDir.ts: BaseAgentDefinition
interface BaseAgentDefinition {
  agentType: string;                        // unique identifier
  whenToUse: string;                         // usage scenario (for model decision-making)
  tools?: string[];                          // allowed tool list, ['*'] = all allowed
  disallowedTools?: string[];                // disallowed tool list
  skills?: string[];                         // preloaded skill list
  mcpServers?: AgentMcpServerSpec[];         // agent-specific MCP servers
  hooks?: HooksSettings;                     // session hooks configuration
  color?: AgentColorName;                    // UI color
  model?: string | 'inherit';               // model override, inherit = inherit from parent
  effort?: EffortValue;                     // effort level marker
  permissionMode?: PermissionMode;           // permission mode
  maxTurns?: number;                         // max conversation turns
  filename?: string;                         // source filename
  baseDir?: string;                          // source directory
  criticalSystemReminder_EXPERIMENTAL?: string; // critical system reminder
  requiredMcpServers?: string[];            // required MCP servers
  background?: boolean;                      // force background execution
  initialPrompt?: string;                   // initial prompt
  memory?: AgentMemoryScope;                 // memory scope
  isolation?: 'worktree' | 'remote';        // isolation mode
  pendingSnapshotUpdate?: { snapshotTimestamp: string }; // snapshot update
  omitClaudeMd?: boolean;                   // skip CLAUDE.md injection
}

// Three Agent sources

// Type 1: CC built-in agents, including:
// Always enabled: generalPurpose, statuslineSetup
// Conditionally enabled: explore, plan, claudeCodeGuide, verification
// When the COORDINATOR_MODE environment variable is enabled, these 6 agents
// are replaced by getCoordinatorAgents() as worker agents in coordinator mode
interface BuiltInAgentDefinition extends BaseAgentDefinition {
  source: 'built-in';
  baseDir: 'built-in';
  getSystemPrompt(params: AgentSystemPromptParams): string;
  callback?(params: AgentCallbackParams): Promise<void> | void;
}

// Type 2: Agents defined by user/project/policy configuration files
interface CustomAgentDefinition extends BaseAgentDefinition {
  source: SettingSource; // 'userSettings' | 'projectSettings' | 'policySettings' |
'flagSettings'
  getSystemPrompt(): string;
  filename?: string;
  baseDir?: string;
}
```

```
// Type 3: Agents registered by external plugin system
interface PluginAgentDefinition extends BaseAgentDefinition {
  source: 'plugin';
  getSystemPrompt(): string;
  filename?: string;
  plugin: string;
}
```

2.3 Agent Tool Call Result

```
// agentToolUtils.ts: AgentToolResult (Zod Schema)
const AgentToolResult = z.object({
  agentId: z.string(), // agent instance ID for this round
  agentType: z.string().optional(), // agent type name (e.g., 'explore', 'plan', etc.)
  content: z.array(z.object({ // content returned by agent, type is currently fixed to
'text'
    type: z.literal('text'),
    text: z.string(), // agent's text output
  })),
  totalToolUseCount: z.number(), // total tool calls within this round
  totalDurationMs: z.number(), // total execution time in milliseconds
  totalTokens: z.number(), // total tokens consumed (input + output)
  usage: z.object({ // detailed usage data from API response
    input_tokens: z.number(), // input tokens for this request
    output_tokens: z.number(), // output tokens for this request
    cache_creation_input_tokens: z.number().nullable(), // input tokens consumed when
creating persistent cache (supported by some models)
    cache_read_input_tokens: z.number().nullable(), // token savings from reading existing
cached context
    server_tool_use: z.object({ // server-side tool call statistics
      web_search_requests: z.number(), // web search request count
      web_fetch_requests: z.number(), // web fetch request count
    }).nullable(),
    service_tier: z.enum(['standard', 'priority', 'batch']).nullable(), // API service tier
    cache_creation: z.object({ // short-term cache (ephemeral cache) usage
      ephemeral_1h_input_tokens: z.number(), // cache consumed tokens with ~1 hour TTL
      ephemeral_5m_input_tokens: z.number(), // cache consumed tokens with ~5 minute TTL
    }).nullable(),
  }),
});
```

2.4 Task State

```
// LocalAgentTask.tsx: LocalAgentTaskState
interface LocalAgentTaskState {
  type: 'local_agent';                          // state machine type marker
  agentId: string;                              // globally unique ID
  prompt: string;                               // task description
  selectedAgent?: AgentDefinition;              // associated agent definition
  agentType: string;                            // agent type string
  model?: string;                               // model used
```

```
  abortController?: AbortController;            // abort controller
  unregisterCleanup?: () => void;               // unregister cleanup function
  error?: string;                               // error message
  result?: AgentToolResult;                     // final result
  progress?: AgentProgress;                     // current progress
  retrieved: boolean;                           // whether UI has read it
  messages?: Message[];                         // message history (in-memory cache)
  lastReportedToolCount: number;                // last reported tool call count
  lastReportedTokenCount: number;               // last reported token count
  isBackgrounded: boolean;                      // whether moved to background
  pendingMessages: string[];                   // pending delivery message queue
  retain: boolean;                              // UI holds reference (prevent GC)
  diskLoaded: boolean;                          // bootstrap completed
  evictAfter?: number;                          // GC deadline timestamp
}

// Progress data structures
interface ToolActivity {
  toolName: string;
  input: Record<string, unknown>;
  activityDescription?: string;
  isSearch?: boolean;       // is this a search tool (low-intensity activity)
  isRead?: boolean;        // is this a read-only tool
}

interface AgentProgress {
  toolUseCount: number;
  tokenCount: number;
  lastActivity?: ToolActivity;
  recentActivities?: ToolActivity[];  // keeps up to 5 entries
  summary?: string;                   // periodic summary
}

// Progress tracker (runtime)
interface ProgressTracker {
  toolUseCount: number;
  latestInputTokens: number;           // cumulative input tokens (takes latest value)
  cumulativeOutputTokens: number;       // cumulative output tokens
  recentActivities: ToolActivity[];      // recent activities (up to 5)
}
```

2.5 Fork-Specific Data Structures

```
// forkedAgent.ts: CacheSafeParams
interface CacheSafeParams {
  systemPrompt: SystemPrompt;
  userContext: { [k: string]: string };
  systemContext: { [k: string]: string };
  toolUseContext: ToolUseContext;
  forkContextMessages: Message[];
}

// forkedAgent.ts: ForkedAgentParams
```

```typescript
interface ForkedAgentParams {
  promptMessages: Message[];
  cacheSafeParams: CacheSafeParams;
  canUseTool: CanUseToolFn;
  querySource: QuerySource;
  forkLabel: string;
  overrides?: SubagentContextOverrides;
  maxOutputTokens?: number;
  maxTurns?: number;
  onMessage?: (message: Message) => void;
  skipTranscript?: boolean;
  skipCacheWrite?: boolean;
}

// forkedAgent.ts: SubagentContextOverrides
interface SubagentContextOverrides {
  options?: ToolUseContext['options'];
  agentId?: string;
  agentType?: string;
  messages?: Message[];
  readFileState?: ToolUseContext['readFileState'];
  abortController?: AbortController;
  getAppState?: ToolUseContext['getAppState'];
  shareSetAppState?: boolean;         // share parent's setAppState
  shareSetResponseLength?: boolean;   // share parent's setResponseLength
  shareAbortController?: boolean;    // share parent's AbortController
  criticalSystemReminder_EXPERIMENTAL?: string;
  requireCanUseTool?: boolean;
  contentReplacementState?: ContentReplacementState;
}
```

2.6 Permission Mode Enum

```typescript
type PermissionMode =
  | 'bubble'           // permission requests bubble up to parent terminal
  | 'plan'             // plan generation and approval required before execution
  | 'acceptEdits'      // auto-approve file modifications
  | 'bypassPermissions'// skip all permission checks
  | 'auto';            // default behavior
```

2.7 Memory Scope

```typescript
type AgentMemoryScope = 'user' | 'project' | 'local';
// user    → ~/.claude/agent-memory/<agentType>/      cross-project global
// project → <cwd>/.claude/agent-memory/<agentType>/   team-shared
// local   → <cwd>/.claude/agent-memory-local/<agentType>/  local only
```

Chapter 3: Agent Creation and Startup Flow

3.1 Complete Decision Tree

```
AgentTool.call(input, toolUseContext, canUseTool, assistantMessage, onProgress?)
│
├─ [Step 1] Permission Check
│   └─ canUseTool('Agent') → verify user is authorized to use Agent tool
│
├─ [Step 2] Multi-Agent Spawn? (team_name + name)
│   └─ Yes → spawnTeammate() → skip AgentTool logic, generate teammate_spawned
│
├─ [Step 3] Resolve Agent Type  ←──────────────────────────────┐
│   ├─ subagent_type specified → use that type                  │
│   ├─ forkEnabled + no type                                    │
│   │   └─ Is Fork Subagent → type = 'fork'                    │
│   └─ none → general-purpose (default)                         │
│                                                               │
├─ [Step 4] Recursive Fork Guard                                │
│   └─ querySource === 'agent:builtin:fork' → reject, prevent infinite recursion │
│                                                               │
├─ [Step 5] MCP Requirements Validation                         │
│   ├─ agent with requiredMcpServers                            │
│   │   ├─ wait for pending servers up to 30 seconds            │
│   │   └─ validateMcpRequirements() → error if missing         │
│   └─ no requirements → skip                                   │
│                                                               │
├─ [Step 6] Isolation Setup                                     │
│   ├─ isolation = 'remote' → delegate to CCR → remote_launched │
│   └─ isolation = 'worktree' → create git worktree             │
│                                                               │
├─ [Step 7] System Prompt Setup                                 │
│   ├─ Fork path → inherit parent's rendered system prompt      │
│   └─ Normal path → getAgentSystemPrompt() → build new prompt  │
│                                                               │
├─ [Step 8] Sync/Async Decision  ──────────────────────────────┘
│   │
│   ├─ [Sync Branch] run_in_background = false
│   │   ├─ registerAgentForeground() → register task, start auto-background countdown
│   │   ├─ onProgress hint after 2s (PROGRESS_THRESHOLD_MS)
│   │   ├─ race: next message vs background signal
│   │   │   ├─ message arrives first → finalizeResult(), unregister → return
│   │   │   └─ signal arrives first → backgroundAgentTask() → async_launched
│   │   └─ error handling → partialResult recovery, AbortError propagated
│   │
│   └─ [Async Branch] run_in_background = true (or coordinator/fork)
│       ├─ registerAsyncAgent() → agentId, task state = pending
│       ├─ agentNameRegistry[name] = agentId (if named)
│       ├─ runAsyncAgentLifecycle() started in background
│       └─ returns { status: 'async_launched', agentId, outputFile, ... }
│
└─ [Step 9] Cleanup (finally block)
    ├─ mcpCleanup()
    ├─ clearSessionHooks()
    ├─ killShellTasksForAgent()
    └─ killMonitorMcpTasksForAgent()
```

3.2 Agent Type Routing Table

`subagent_type`	Agent Definition Source	Execution Mode	Default Tools
`"Explore"`	Built-in	sync one-shot	read-only
`"Plan"`	Built-in	sync one-shot	read-only
`"general-purpose"`	Built-in	sync/async	all allowed
`"worker"`	Built-in (coordinator)	async	restricted set
`"fork"`	virtual type (Fork)	async	inherits parent
user-defined name	md/json/plugins	sync/async	as defined
unspecified	`general-purpose`	sync→async	all allowed

3.3 Async vs Sync Decision Algorithm

```
// AgentTool.tsx: determineSyncAsync logic
function shouldRunAsync(options: AgentToolInput, agentType: string): boolean {
  // explicitly requests background
  if (options.run_in_background) return true;

  // subagent in coordinator mode
  if (isCoordinatorMode()) return true;

  // Fork subagent
  if (isForkSubagentEnabled() && !options.subagent_type) return true;

  // agent definition requires background
  const agentDef = getAgentDefinition(agentType);
  if (agentDef?.background) return true;

  return false;
}
```

3.4 Auto-Background Mechanism

Sync agents have a 2-second delay hint (`PROGRESS_THRESHOLD_MS = 2000`), after which they automatically move to background:

```
// sync branch
const { taskId, backgroundSignal, cancelAutoBackground } =
  registerAgentForeground({ agentId, prompt, agentType, model, abortController });

// after 2 seconds, show background hint
const autoBackgroundTimer = setTimeout(() => {
  // parent session UI shows background hint
  showBackgroundHint(agentType, agentId, outputFile);
}, PROGRESS_THRESHOLD_MS);

// race: message completion vs background signal
const raceResult = await Promise.race([
  waitForNextMessage(),
  backgroundSignal,
```

```
]);

if (raceResult === 'backgrounded') {
  // background signal arrived first → convert to async
  backgroundAgentTask(taskId, getAppState, setAppState);
  return { status: 'async_launched', agentId, outputFile, ... };
} else {
  // message completed
  cancelAutoBackground();
  return finalizeResult(messages);
}
```

Chapter 4: Message Loop and Query Engine

4.1 Complete runAgent() Execution Flow

```
// runAgent.ts: runAgent() pseudocode
async function* runAgent(params: RunAgentParams): AsyncGenerator<Message> {

  // ── Phase 1: Initialization ──────────────────────────────────
  const agentDef = getAgentDefinition(params.agentType);
  const model = resolveAgentModel(agentDef, params.model);

  // generate agentId (if not provided)
  const agentId = params.agentId ?? generateAgentId();

  // register performance tracking
  Perfstart('agent', agentId, agentDef.agentType);

  // ── Phase 2: Build Context ───────────────────────────────────
  // 2a. build user/system context
  const { userContext, systemContext } = buildAgentContext({
    omitClaudeMd: agentDef.omitClaudeMd,
    cwd: params.cwd,
  });

  // 2b. build permission context
  const permissionContext = buildPermissionContext({
    parentMode: toolUseContext.options?.permissionMode,
    agentMode: agentDef.permissionMode,
    // special rule: avoid permission popups for async/bypassPermissions
    shouldAvoidPermissionPrompts:
      params.isAsync ||
      agentDef.permissionMode === 'bypassPermissions',
    awaitAutomatedChecksBeforeDialog:
      !params.isAsync && agentDef.permissionMode === 'acceptEdits',
  });

  // 2c. resolve tool set
  const tools = params.useExactTools
    ? toolUseContext.tools  // Fork path: fully inherit parent's tools
    : resolveAgentTools(agentDef, availableTools, params.isAsync);
```

```
// 2d. build system prompt
const systemPromptLines = await getAgentSystemPrompt(
  agentDef, toolUseContext, model, dirs, tools
);

// ── Phase 3: Pre-Hook ──────────────────────────────────────────
// 3a. execute SubagentStart hooks
await executeHooks('SubagentStart', { agentDef, agentId });

// 3b. register frontmatter hooks (from agent definition)
registerFrontmatterHooks(agentDef.hooks);

// 3c. preload skills
await preloadSkills(agentDef.skills ?? [], toolUseContext);

// ── Phase 4: Agent MCP Servers ─────────────────────────────────
const { clients: agentMcpClients, tools: agentMcpTools, cleanup: mcpCleanup } =
  await initializeAgentMcpServers(agentDef, toolUseContext.mcpClients);

// ── Phase 5: Create Isolation Context ──────────────────────────
const subagentContext = createSubagentContext(toolUseContext, {
  agentId,
  agentType: agentDef.agentType,
  // AbortController linked to parent
  abortController: createChildAbortController(
    toolUseContext.abortController,
    agentId
  ),
  // Async agent cannot control parent UI
  setAppState: undefined,
  setResponseLength: undefined,
  // read state cache clone (preserves parent's cache hit decisions)
  readFileState: toolUseContext.readFileState,
});

// ── Phase 6: Record Initial Transcript ─────────────────────────
recordSidechainTranscript({
  type: 'metadata',
  agentId,
  agentType: agentDef.agentType,
  model,
  systemPromptLines,
  startTime: Date.now(),
});

// ── Phase 7: Main Query Loop ───────────────────────────────────
try {
  for await (const message of query({
    // ... query options
  })) {
    // forward TTFT/OTPS metrics
    if (message.stream_event?.type === 'stream_request_start') {
```

```
        const { ttft_ms, first_token_ms } = message.stream_event;
        forwardMetric('time_to_first_token_ms', ttft_ms);
        forwardMetric('output_tokens_per_second', calculateOTPS(first_token_ms));
      }

      // filter incomplete tool_use (tail truncation protection)
      if (message.type === 'assistant' && isMaxTurnsReached(message)) {
        yield* emitIncompleteToolCallWarning(message, tools);
      }

      // record to transcript file
      if (isRecordableMessage(message)) {
        recordSidechainTranscript(message);
      }

      // append to agentMessages (for final result)
      agentMessages.push(message);

      yield message;  // send to parent session
    }
  }

  // ── Phase 8: Cleanup ──────────────────────────────────────────
  finally {
    await mcpCleanup();                        // disconnect Agent MCP
    clearSessionHooks();                       // clear Hooks
    releasePromptCacheTracking(agentId);       // release Prompt Cache
    releaseFileStateCache();                  // release file read cache
    Perfstop('agent', agentId);               // stop performance tracking
    clearSubagentTranscriptDir();             // clear transcript temp files
    removeTodoEntry(agentId);                 // clear Todo entries
    killShellTasksForAgent(agentId);          // kill Agent's Shell
    killMonitorMcpTasksForAgent(agentId);     // stop MCP monitor tasks
  }
}
```

4.2 forkSubagent Message Building Algorithm

```
// forkSubagent.ts: buildForkedMessages()
// Goal: build a message sequence byte-identical to the parent's API request
// Purpose: maximize API cache hit (response_prefix can be reused)

function buildForkedMessages(
  directive: string,          // child agent's task directive
  assistantMessage: AssistantMessage  // parent's last assistant message
): MessageType[] {
  // Step 1: deep copy to avoid polluting parent state
  const clonedAssistant = deepClone(assistantMessage);

  // Step 2: collect all tool_use blocks
  const toolUses = clonedAssistant.content
    .filter((block): block is ToolUseBlock => block.type === 'tool_use');
```

```
  // Step 3: create byte-identical placeholder results for each tool_use
  // Key: text must be exactly the same → API can hit cache
  const toolResultBlocks: ToolResultBlock[] = toolUses.map(toolUse => ({
    type: 'tool_result' as const,
    tool_use_id: toolUse.id,
    content: [{
      type: 'text' as const,
      text: FORK_PLACEHOLDER_RESULT,  // ≡ 'Fork started — processing in background'
      is_error: false,
    }],
  }));

  // Step 4: build user directive message
  // Note: this message is new (not part of cache), but doesn't affect cache hit rate
  const userDirectiveMessage: Message = {
    type: 'user',
    content: [{
      type: 'text',
      text: buildChildMessage(directive),  // contains 10 rules
    }],
  };

  // Step 5: assemble final message sequence
  return [
    clonedAssistant,            // 1. parent's assistant (with tool_use blocks)
    ...toolResultBlocks,         // 2. N identical placeholder tool_results
    userDirectiveMessage,        // 3. directive message (triggers child agent execution)
  ];
}

// buildChildMessage: generate XML directive block
function buildChildMessage(directive: string): string {
  return `<${FORK_BOILERPLATE_TAG}>
  Task: ${directive}
  Rules:
  1. Continue from the previous agent's last action
  2. Complete the task autonomously — do not re-confirm
  3. Report completion with a concise summary
  4. For multi-step: execute all remaining steps
  5. One-shot: run to completion without asking
  6. Tool restrictions override your default capabilities
  7. If blocked: attempt alternatives, report if impossible
  8. Do not explain your reasoning unless asked
  9. Result goes to parent session — avoid redundant context
  10. You control execution — parent is waiting for result
</${FORK_BOILERPLATE_TAG}>`;
}
```

4.3 Async Agent Lifecycle

```
// agentToolUtils.ts: runAsyncAgentLifecycle()
async function runAsyncAgentLifecycle(params: {
  agentId: string;
```

```
  toolUseContext: ToolUseContext;
  onProgress?: (message: Message) => void;
  enableSummarization?: boolean;
  ...others
}): Promise<void> {

  // Step 1: create progress tracker
  const tracker = createProgressTracker();
  const resolveActivityDescription = createActivityDescriptionResolver(
    toolUseContext.tools ?? []
  );

  const agentMessages: Message[] = [];

  // Step 2: main loop — consume messages from runAgent()
  try {
    for await (const message of runAgent({ agentId, ... })) {
      // save message history
      agentMessages.push(message);

      // persist to disk (for Bootstrap / TaskOutput)
      if (message.type === 'assistant' || message.type === 'progress') {
        rootSetAppState(draft => {
          const task = draft.localAgentTasks[agentId];
          if (task?.retain) {
            task.messages = [...(task.messages ?? []), message];
          }
        });
      }

      // update progress tracker
      updateProgressFromMessage(tracker, message, resolveActivityDescription);

      // send progress event to parent session
      emitTaskProgress(
        tracker, agentId,
        message.type === 'assistant'
          ? getLastToolUseId(message)
          : undefined,
        resolveActivityDescription(message),
        startTime,
        getLastToolUseName(message)
      );

      // callback notification
      params.onProgress?.(message);
    }

    // Step 3: build result
    const result = finalizeAgentTool(agentMessages, agentId, {
      startTime,
      totalToolUseCount: tracker.toolUseCount,
      // ... usage metrics
```

```
    });

    // Step 4: stop summarization (if enabled)
    if (params.enableSummarization) {
      await stopAgentSummarization(agentId);
    }

    // Step 5: mark complete (mark first, ensure state consistency)
    completeAgentTask(result, setAppState);

    // Step 6: security review
    await classifyHandoffIfNeeded({ agentId, result, toolUseContext });

    // Step 7: send notification
    enqueueAgentNotification({
      agentId,
      toolUseId: params.toolUseId,
      status: 'completed',
      result,
      worktree: params.getWorktreeResult?.(),
      usage: result.usage,
    });

  } catch (error) {
    if (isAbortError(error)) {
      // user abort: send killed notification (not failed)
      enqueueAgentNotification({
        agentId, status: 'killed',
        result: extractPartialResult(agentMessages),
      });
    } else {
      // real error
      failAgentTask(agentId, String(error), setAppState);
      enqueueAgentNotification({
        agentId, status: 'failed',
        result: extractPartialResult(agentMessages),
        error: String(error),
      });
    }
  } finally {
    // Step 8: cleanup
    await clearInvokedSkills(agentId);
    await dumpState(agentId);
  }
}
```

Chapter 5: Tool Filtering and Permission System

5.1 Six-Layer Filtering Algorithm

```
// agentToolUtils.ts: filterToolsForAgent()
// Input: all tools, agent definition, isAsync, permission mode
```

```
// Output: filtered tool set

function filterToolsForAgent(
  allTools: Tools,
  agentDef: AgentDefinition,
  isAsync: boolean = false,
  permissionMode: PermissionMode = 'auto'
): Tools {

  let result = allTools;

  // ── Layer 1: Global Blocklist ──────────────────────────────
  // ALL_AGENT_DISALLOWED_TOOLS: disabled for all agent types
  result = result.exclude(ALL_AGENT_DISALLOWED_TOOLS);
  // Agent nesting exception: only allowed in ant internal builds (prevent external
recursive calls)
  if (process.env.USER_TYPE !== 'ant') {
    result = result.exclude('Agent');
  }

  // ── Layer 2: Async Agent Mandatory Allowlist ────────────────
  // async background agents cannot use dangerous tools
  if (isAsync) {
    result = result.filter(t =>
      ASYNC_AGENT_ALLOWED_TOOLS.has(t.name)
    );
  }

  // ── Layer 3: Custom Agent Extra Blocks ──────────────────────
  // custom agents (md/json config) additionally block task management tools
  if (agentDef.source === 'userSettings' ||
      agentDef.source === 'projectSettings') {
    result = result.exclude(CUSTOM_AGENT_DISALLOWED_TOOLS);
  }

  // ── Layer 4: Agent Definition Allowlist ─────────────────────
  // ['*'] means all allowed (but still subject to Layer 1/2/3)
  if (agentDef.tools && !agentDef.tools.includes('*')) {
    result = result.filter(t => agentDef.tools!.includes(t.name));
  }

  // ── Layer 5: Agent Definition Blocklist (higher priority) ───
  if (agentDef.disallowedTools) {
    result = result.exclude(agentDef.disallowedTools);
  }

  // ── Layer 6: Permission Mode Adjustment ─────────────────────
  switch (permissionMode) {
    case 'bypassPermissions':
      // remove all permission check wrappers
      result = result.map(t => t.removePermissionWrapper());
      break;
    case 'acceptEdits':
```

```
      // ensure Edit/Write tools are available (but keep confirmation dialogs)
      result = result.map(t => {
        if (t.name === 'Edit' || t.name === 'Write') {
          return t.withAutoApprove();
        }
        return t;
      });
      break;
    case 'plan':
      // all tools require plan first
      result = result.map(t => t.wrapWithPlanApproval());
      break;
  }

  return result;
}
```

5.2 Tool Disallow Sets Explained

```
// constants/tools.ts

// ── All Agents Blocked ──────────────────────────────────────────
const ALL_AGENT_DISALLOWED_TOOLS = new Set([
  TASK_OUTPUT_TOOL_NAME,       // prohibit waiting on other tasks (prevent deadlock)
  EXIT_PLAN_MODE_V2_TOOL_NAME, // prohibit主动 exiting plan mode
  ENTER_PLAN_MODE_TOOL_NAME,  // prohibit主动 entering plan mode
  AGENT_TOOL_NAME,             // prohibit nested Agents (unless ant internal)
  ASK_USER_QUESTION_TOOL_NAME, // prohibit asking user questions (one-way comm)
  TASK_STOP_TOOL_NAME,        // prohibit stopping tasks
  WORKFLOW_TOOL_NAME,         // prohibit triggering workflows (if feature enabled)
]);

// ── Async Background Agents Only Allowed ────────────────────────
const ASYNC_AGENT_ALLOWED_TOOLS = new Set([
  // file operations
  'Read', 'Edit', 'Write', 'NotebookEdit',
  // search tools
  'Grep', 'Glob', 'WebSearch', 'WebFetch',
  // shell execution
  'Bash', ...ALL_SHELL_TOOL_NAMES,
  // task management
  'TodoWrite', 'TodoGet', 'TodoList', 'TodoUpdate',
  // other
  'Skill', 'SyntheticOutput',
  'ToolSearch',
  'EnterWorktree', 'ExitWorktree',
]);

// ── In-process Teammate Extra Allowlist ─────────────────────────
const IN_PROCESS_TEAMMATE_ALLOWED_TOOLS = new Set([
  ...IN_ASYNC_AGENT_ALLOWED_TOOLS,
  'TaskCreate', 'TaskGet', 'TaskList', 'TaskUpdate', 'TaskDelete',
  'SendMessage',
```

```
  // Cron tools (if AGENT_TRIGGERS feature enabled)
  ...(FEATURES.AGENT_TRIGGERS ? ['CronCreate', 'CronDelete', 'CronList'] : []),
]);

// ── Coordinator Mode Agent ──────────────────────────────────────
const COORDINATOR_MODE_ALLOWED_TOOLS = new Set([
  'Agent', 'TaskStop', 'SendMessage', 'SyntheticOutput',
  'Read', 'Write', 'Edit', 'Bash',
  'Grep', 'Glob', 'WebSearch', 'WebFetch',
  // ... most tools allowed
]);
```

5.3 Tool Filtering Timing and resolveAgentTools

```
// agentToolUtils.ts: resolveAgentTools()
// works with filterToolsForAgent, handles wildcard expansion

function resolveAgentTools(
  agentDef: AgentDefinition,
  availableTools: Tools,
  isAsync?: boolean,
  isMainThread?: boolean,
): ResolvedAgentTools {

  // Case 1: ['*'] wildcard → expand to all available tools
  if (agentDef.tools?.includes('*')) {
    return {
      hasWildcard: true,
      validTools: availableTools.map(t => t.name),
      invalidTools: [],
      resolvedTools: filterToolsForAgent(
        availableTools, agentDef, isAsync, permissionMode
      ),
      allowedAgentTypes: undefined,
    };
  }

  // Case 2: explicit list → verify each tool exists
  const validTools: string[] = [];
  const invalidTools: string[] = [];

  for (const toolName of (agentDef.tools ?? [])) {
    if (availableTools.has(toolName)) {
      validTools.push(toolName);
    } else {
      invalidTools.push(toolName);  // tool doesn't exist → warning
    }
  }

  return {
    hasWildcard: false,
    validTools,
    invalidTools,
```

```
    resolvedTools: filterToolsForAgent(validTools, agentDef, isAsync),
    allowedAgentTypes: undefined,
  };
}
```

Chapter 6: Fork Subagent Algorithm

6.1 Fork vs Normal Subagent Core Differences

Dimension	Normal Subagent	Fork Subagent
Trigger condition	explicitly specify `subagent_type`	`forkEnabled=true` + no type
System prompt	newly built (re-tokenize)	byte-level inheritance (cache hit possible)
Tool set	re-filtered	`useExactTools=true` fully inherited
Message history	only `prompt`	complete parent conversation history
API cache hit rate	low (new prefix each time)	high (response_prefix reusable)
Execution mode	sync or async	forced async
Tool call serialization	re-serialized	inherits parent's serialization (byte-identical)
Use case	completely different tasks	continuation/partition of same task

6.2 Recursion Depth Guard

```
// forkSubagent.ts: isInForkChild()
// prevent fork subagent from forking again (infinite recursion)

function isInForkChild(messages: MessageType[]): boolean {
  // scan message history for FORK_BOILERPLATE_TAG XML marker
  // if present, indicates we're in a fork child context
  return messages.some(msg => {
    if (msg.type !== 'user') return false;
    const text = extractTextContent(msg);
    return text.includes(`<${FORK_BOILERPLATE_TAG}>`);
  });
}

// call site: Step 4 in AgentTool.call()
if (
  toolUseContext.querySource === 'agent:builtin:fork' ||
  isInForkChild(assistantMessage)
) {
  throw new Error(
    `Cannot spawn fork sub-agent: Already in fork child context. ` +
    `Fork subagents cannot be nested.`
  );
}
```

Note

When a child Agent is created via the fork mechanism, it fully inherits all the parent's tool capabilities, including the Agent tool itself. Therefore, the child theoretically has the ability to fork a new child again —

which would create an infinite nested recursion. To prevent this, the system injects a unique `<FORK_BOILERPLATE_TAG>` XML marker into the child Agent's context during the fork message injection phase. At the same time, a guard check is placed at the Agent tool entry point: if the marker is already found in the message history (meaning we're already in a fork child context), fork is rejected outright, locking the recursion depth at 1.

6.3 Fork Cache Optimization Principle

```
┌──────────────────────────────────────────────────────────────────┐
│                      API Cache Hit Mechanism                     │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  Parent Agent Request:                                           │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │ System Prompt Hash (SHA256)  ←── parent's system prompt│   │
│  │ Tool Definitions Hash (SHA256) ←── parent's tool ser.   │   │
│  │ Messages Prefix  ←── parent's message history             │   │
│  │ [Assistant Message + tool_use blocks]                    │   │
│  └──────────────────────────────────────────────────────────┘   │
│                           │                                      │
│                           ▼                                      │
│  API Return (cached prefix):                                     │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │ Thinking... [complete thinking text, can be reused]      │   │
│  └──────────────────────────────────────────────────────────┘   │
│                                                                  │
│  Fork Subagent Request:                                          │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │ System Prompt Hash  ←── identical to parent ✓            │   │
│  │ Tool Definitions Hash ←── identical to parent ✓          │   │
│  │ Messages Prefix  ←── parent's assistant + placeholders ✓ │   │
│  │ [Assistant + tool_result placeholders + directive]       │   │
│  └──────────────────────────────────────────────────────────┘   │
│                           │                                      │
│                           ▼                                      │
│  API Return (cache hit):                                         │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │ HTTP 200 + cache: true                                   │   │
│  │ response_prefix_bytes_reused: ~90%                       │   │
│  │ only need to generate completion for new directive msg    │   │
│  └──────────────────────────────────────────────────────────┘   │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

6.4 Fork Worktree Isolation

```
// forkSubagent.ts: buildWorktreeNotice()
// When fork + worktree, send path notice to parent session

function buildWorktreeNotice(
  parentCwd: string,     // /repo
```

```
  worktreeCwd: string,    // /repo/.claude/worktrees/agent-abc
): string {
  return [
    `Note: This agent is running in an isolated git worktree:`,
    `  Worktree: ${worktreeCwd}`,
    `  Parent:   ${parentCwd}`,
    ``,
    `File paths in the worktree map to the parent as follows:`,
    `  ${worktreeCwd}/src/file.ts → ${parentCwd}/src/file.ts`,
    ``,
    `Any changes made here are isolated from the parent branch.`,
    `When complete, these changes will be summarized for you.`,
  ].join('\n');
}
```

Note

- When forking a child Agent, if `isolation: 'worktree'` is specified (running in an isolated git worktree), the child Agent's working directory switches from /repo to an isolated /repo/.claude/worktrees/agent-abc directory. In this case, all file reads/writes by the child Agent are performed in the worktree副本, completely isolated from the parent session's main branch.
- So that the parent session can understand which files the child Agent operated on after completion, the system injects an explanation into the parent session via `buildWorktreeNotice()`, informing: how the paths in the worktree correspond one-to-one to the parent directory (e.g., worktree/src/file.ts maps to 父目录/src/file.ts), and that these modifications are isolated and won't affect the parent branch. This way, after receiving the child Agent's result, the parent session can accurately map changes in the worktree back to the original paths and summarize for the user.

Chapter 7: Built-in Agent Details

Note

Reasons for setting up sub-agents:

- Need to handle **independent multi-step tasks**
- Prevent context overflow/pollution
- Tool and permission isolation (e.g., read-only mode below)
- Can run in parallel or in background asynchronously
- Introduce verification and reflection mechanisms (e.g., `verification`)
- Most common: role specialization and prompt focus

7.1 All Built-in Agents Overview

Name	File	Purpose	Tool Strategy	Mode
`general-purpose`	`generalPurposeAgent.ts`	general tasks	`['*']`	sync/async
`Explore`	`exploreAgent.ts`	fast search	read-only	one-shot
`Plan`	`planAgent.ts`	architecture design	read-only	one-shot

Name	File	Purpose	Tool Strategy	Mode
`verification`	`verificationAgent.ts`	test verification	no edits	async background
`claude-code-guide`	`claudeCodeGuideAgent.ts`	usage help	read/WebSearch	sync
`statusline-setup`	`statuslineSetup.ts`	status bar	all	sync

7.2 ExploreAgent (Fast Exploration)

```
// built-in/exploreAgent.ts
const EXPLORE_AGENT: BuiltInAgentDefinition = {
  source: 'built-in',
  agentType: 'Explore',
  whenToUse: 'Use when you need to quickly search, read, or explore ' +
    'files, code, documentation, or web content.',
  tools: ['*'],              // all allowed (but ExploreSystemPrompt restricts behavior)
  model: isAntInternalUser ? 'inherit' : 'haiku',
  // key: skip CLAUDE.md injection to save context
  omitClaudeMd: true,
  maxTurns: 10,              // limit max turns to prevent excessive exploration
  getSystemPrompt() {
    return `You are Claude Code's fast exploration specialist.

KEY BEHAVIOR:
- Be FAST and DIRECT — minimize reasoning, maximize action
- One-shot execution: read, search, or grep, then report
- Avoid unnecessary tool calls — each call must be purposeful
- Report findings concisely — bullet points preferred
- Do NOT edit, write, or modify files
- Do NOT explain your approach unless asked
- If blocked: report what you found and why you're blocked

TOOL USAGE:
- Read: targeted file reads with specific line ranges
- Grep: precise patterns, limit scope, avoid broad searches
- Glob: specific file patterns, not recursive wildcards
- WebSearch/WebFetch: only for external info not in codebase
- Glob + Read is usually faster than Grep for finding files`;
  },
};
```

7.3 PlanAgent (Architecture Planning)

```
// built-in/planAgent.ts
const PLAN_AGENT: BuiltInAgentDefinition = {
  source: 'built-in',
  agentType: 'Plan',
  whenToUse: 'Use when you need to design an architecture, ' +
    'plan a refactor, or create a detailed implementation plan.',
  tools: ['*'],              // all allowed (but plan prompt restricts modifications)
  model: isAntInternalUser ? 'inherit' : 'haiku',
  omitClaudeMd: true,         // no need for CLAUDE.md context
  maxTurns: 20,               // planning needs more turns
```

```
  getSystemPrompt() {
    return `You are Claude Code's architecture and planning specialist.

CRITICAL: READ-ONLY MODE — You must NOT modify any files.

YOUR ROLE:
- Analyze requirements and design solutions
- Create detailed implementation plans (Markdown)
- Identify risks, dependencies, and trade-offs
- Suggest file changes ONLY as text descriptions

OUTPUT FORMAT:
## Analysis
[What the task requires]

## Proposed Solution
[High-level approach]

## Implementation Plan
### Step 1: [Name]
- File: ...
- Changes: ...
[Continue for each step]

## Risks & Mitigations
[Potential issues and how to address them]

## Alternative Approaches
[Brief alternatives with trade-offs]

TOOL USAGE:
- Read: understand existing code structure
- Grep/Glob: find relevant files
- WebSearch: research best practices or APIs
- WebFetch: reference external documentation`;
  },
};
```

7.4 VerificationAgent (Verification)

```
// built-in/verificationAgent.ts
const VERIFICATION_AGENT: BuiltInAgentDefinition = {
  source: 'built-in',
  agentType: 'verification',
  whenToUse: 'Use when you need to verify code correctness, ' +
    'run tests, or perform quality assurance checks.',
  tools: ['*'],
  model: 'sonnet',
  background: true,           // force background execution
  maxTurns: 50,
  getSystemPrompt() {
    return `You are Claude Code's verification and QA specialist.
```

```
YOUR MISSION:
Think like an adversarial reviewer. Find what could break.

VERIFICATION CATEGORIES (apply all relevant):
1. Frontend: render check, interaction, responsive
2. Backend: API correctness, error handling, edge cases
3. CLI: argument parsing, output format, exit codes
4. Infrastructure: config, secrets, scaling, security
5. Library: API surface, types, exports, documentation
6. Mobile: platform conventions, lifecycle
7. Data/ML: training, inference, data pipeline
8. Database: migrations, queries, indexes, constraints
9. Refactoring: behavior preservation, backward compatibility

REQUIRED OUTPUT FORMAT:
## Verifications Performed
[What you tested]

## Issues Found
### [Severity] - [Category]
**File:** \`path/to/file\`
**Problem:** [Description]
**Impact:** [What breaks]

## VERDICT
\`\`\`
PASS: All checks passed
PARTIAL: N issues found (see above)
FAIL: Critical issues prevent merge
\`\`\`

ANTI-PATTERNS TO AVOID:
- "This looks fine" (without checking)
- Skipping edge cases
- Assuming happy paths
- Not running the actual tests`;
  },
};
```

7.5 ClaudeCodeGuideAgent (Usage Guide)

```
// built-in/claudeCodeGuideAgent.ts
// dynamically builds system prompt with real tool usage context

interface ClaudeCodeGuideParams {
  availableSkills: string[];
  availableAgents: string[];
  availableMcpServers: string[];
  availablePlugins: string[];
  installedCommands: string[];
  uses3PServices: boolean;  // whether third-party services are used
}
```

```
function getClaudeCodeGuideSystemPrompt(params: ClaudeCodeGuideParams): string {
  return [
    `You help users with Claude Code (the CLI), Claude Agent SDK, and Claude API.`,
    ``,
    `## Your Context`,
    // dynamically inject tools the user actually has access to
    `Available skills: ${params.availableSkills.join(', ') || 'none'}`,
    `Available agents: ${params.availableAgents.join(', ') || 'built-in only'}`,
    `Available MCP servers: ${params.availableMcpServers.join(', ') || 'none'}`,
    `Available plugins: ${params.availablePlugins.join(', ') || 'none'}`,
    `Installed CLI commands: ${params.installedCommands.join(', ')}`,
    ``,
    // adjust feedback style based on usage
    params.uses3PServices
      ? `Note: For external service issues, recommend checking official docs.`
      : ``,
  ].join('\n');
}
```

7.6 Feature Flags

```
// builtInAgents.ts: conditional registration
function getBuiltInAgents(): AgentDefinition[] {
  const agents = [GENERAL_PURPOSE_AGENT, STATUSLINE_SETUP_AGENT];

  // Explore/Plan agents: controlled by BUILTIN_EXPLORE_PLAN_AGENTS feature flag
  if (areExplorePlanAgentsEnabled()) {
    agents.push(EXPLORE_AGENT, PLAN_AGENT);
  }

  // Verification agent: controlled by GrowthBook experiment
  if (isVerificationAgentEnabled()) {
    agents.push(VERIFICATION_AGENT);
  }

  // Claude Code Guide: non-SDK environments
  if (!isSdkEntrypoint()) {
    agents.push(CLAUDE_CODE_GUIDE_AGENT);
  }

  // coordinator mode: use different worker agents
  if (isCoordinatorMode()) {
    return getCoordinatorAgents();  // from coordinatorMode.ts
  }

  return agents;
}
```

Chapter 8: Persistent Memory System

Note

See the "Memory System" documentation for details

8.1 Three-Tier Memory Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                    Agent Memory Architecture                     │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  scope = 'user'                                                  │
│  path: ~/.claude/agent-memory/<agentType>/                       │
│  content: ~/.claude/projects/-home-xxx/MEMORY.md                 │
│  access: all projects, all sessions                              │
│  use: user preferences, work style, long-term constraints        │
│                                                                  │
│  scope = 'project'                                               │
│  path: <cwd>/.claude/agent-memory/<agentType>/                   │
│  content: <cwd>/.claude/agent-memory/MEMORY.md                   │
│  access: all sessions within current project                     │
│  use: project context, team conventions, issue tracking          │
│                                                                  │
│  scope = 'local'                                                 │
│  path: <cwd>/.claude/agent-memory-local/<agentType>/             │
│  content: <cwd>/.claude/agent-memory-local/MEMORY.md             │
│  access: current task only                                       │
│  use: one-off task state, temporary reasoning results            │
│                                                                  │
│  Memory format:                                                  │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │ ---                                                      │    │
│  │ name: {{memory name}}                                    │    │
│  │ description: {{one-line description}}                    │    │
│  │ type: {{user, feedback, project, reference}}             │    │
│  │ ---                                                      │    │
│  │                                                          │    │
│  │ {{memory content}}                                       │    │
│  └──────────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────┘
```

8.2 Memory Loading Algorithm

```
// agentMemory.ts: loadAgentMemoryPrompt()

function loadAgentMemoryPrompt(
  agentType: string,
  scope: AgentMemoryScope
): string {
  // Step 1: get memory file path
  const memoryPath = getAgentMemoryEntrypoint(agentType, scope);

  // Step 2: read file content
  if (!existsSync(memoryPath)) {
```

```
    return '';  // no memory file → return empty
  }

  const content = readFileSync(memoryPath, 'utf-8');

  // Step 3: parse frontmatter
  const { metadata, body } = parseFrontmatter(content);

  // Step 4: filter expired memories (if decay field exists)
  const validMemories = filterValidMemories(metadata, body);

  // Step 5: build injection text
  return [
    `## Relevant Memory`,
    `(Scope: ${getMemoryScopeDisplay(scope)})`,
    ``,
    ...validMemories.map(m => `- ${m.description}\n  ${m.body}`),
  ].join('\n');
}

// usage: in buildAgentSystemPrompt()
function getAgentSystemPrompt(agentDef: AgentDefinition, ...) {
  const systemPromptLines: string[] = [];

  // agent's built-in system prompt
  if (isBuiltInAgent(agentDef)) {
    systemPromptLines.push(agentDef.getSystemPrompt(params));
  } else {
    systemPromptLines.push(agentDef.getSystemPrompt());
  }

  // inject memory (if memory scope exists)
  if (agentDef.memory) {
    const memoryPrompt = loadAgentMemoryPrompt(
      agentDef.agentType,
      agentDef.memory
    );
    if (memoryPrompt) {
      systemPromptLines.push(memoryPrompt);
    }
  }

  return systemPromptLines;
}
```

8.3 Memory Safe Path Validation

```
// agentMemory.ts: isAgentMemoryPath()

// prevent path traversal attacks: ensure memory access stays within bounds
function isAgentMemoryPath(absolutePath: string): boolean {
  const normalized = path.normalize(absolutePath);
```

```
  // check three base paths
  const checks = [
    normalized.startsWith(path.join(os.homedir(), '.claude', 'agent-memory')),
    normalized.startsWith(path.join(process.cwd(), '.claude', 'agent-memory')),
    // support remote mount paths
    normalized.startsWith(process.env.CLAUDE_CODE_REMOTE_MEMORY_DIR ?? ''),
  ];

  return checks.some(Boolean);
}
```

Chapter 9: Task State Machine and Lifecycle

9.1 Complete State Machine

```
                          ┌──────────────────────────┐
                          │         pending          │
                          │   (registered, waiting   │
                          │    to start)             │
                          │   registerAsyncAgent()   │
                          └──────────────────────────┘
                                       │
                                       │
                       registerAgentForeground()
                       (sync branch registration)
                                       │
                                       ▼
                          ┌──────────────────────────┐
                          │          running         │
                          │    (agent executing)     │
                          │   runAgent() / query()   │
                          └──────────────────────────┘
                                       │
                                       │
              ┌────────────────────────┼────────────────────────┐
              │                        │                        │
       external abort (ESC)    timeout / maxTurns    normal completion
              │                        │                        │
              ▼                        ▼                        ▼
     ┌──────────────────┐     ┌──────────────────┐   ┌──────────────────┐
     │      killed      │     │     completed    │   │     completed    │
     │   (user abort)   │     │   (with partial  │   │   (complete      │
     │                  │     │    result)       │   │    result)       │
     └──────────────────┘     └──────────────────┘   └──────────────────┘

                                       ▲
                                       │
                          async loop exception / error
                                       │
                                       ▼
                          ┌──────────────────────────┐
                          │         failed           │
                          │    (execution error)     │
```

```
                                  │  runAsyncAgentLifecycle  │
                                  └──────────────────────────┘
```

9.2 Notification XML Format

```
<!-- enqueueAgentNotification() generated format -->

<task-notification>
  <task-id>agent-7f3a9c</task-id>
  <output-file>
    /home/XXXX/.claude/agent-transcripts/agent-7f3a9c.jsonl
  </output-file>
  <tool-use-id>agent-tool-use-7f3a9c</tool-use-id>
  <status>completed</status>
  <summary>Agent "Explore" completed</summary>
  <result>
    Found 12 TypeScript files matching the pattern:
    - src/auth/login.ts
    - src/auth/session.ts
    ...
  </result>
  <usage>
    <total_tokens>4820</total_tokens>
    <input_tokens>1204</input_tokens>
    <output_tokens>3616</output_tokens>
    <tool_uses>7</tool_uses>
    <duration_ms>8430</duration_ms>
    <cache_creation_input_tokens>14200</cache_creation_input_tokens>
    <cache_read_input_tokens>3200</cache_read_input_tokens>
  </usage>
  <!-- only present for worktree isolation -->
  <worktree>
    <worktree-path>
      /repo/.claude/worktrees/agent-7f3a9c
    </worktree-path>
    <worktree-branch>claude/agent-7f3a9c</worktree-branch>
  </worktree>
</task-notification>
```

9.3 GC Eviction Policy

When an Agent task completes, the system needs to decide when to remove it from memory (evict) to avoid resource leaks.

Eviction requires all four conditions to be met simultaneously:

- Condition 1: retain is false, meaning the UI layer no longer holds this task
- Condition 2: diskLoaded is true, meaning the task has been loaded from disk (bootstrap completed)
- Condition 3: task status is completed / failed / killed
- Condition 4: current time has passed the eviction deadline evictAfter

Eviction deadline setting strategy varies by task type:

- Background tasks (Agent running in background by user): allowed to live for 1 hour for easy switching back
- Read foreground tasks: evictable after 5 minutes since user has seen the result
- Active tasks: never evict, evictAfter stays undefined

```
// LocalAgentTask.tsx: task eviction algorithm

function shouldEvictTask(task: LocalAgentTaskState, now: number): boolean {
  return (
    // condition 1: UI doesn't hold
    !task.retain &&
    // condition 2: has been read (bootstrap complete)
    task.diskLoaded &&
    // condition 3: completed / failed / killed
    (task.status === 'completed' ||
     task.status === 'failed' ||
     task.status === 'killed') &&
    // condition 4: past eviction deadline
    (task.evictAfter ? now > task.evictAfter : false)
  );
}

// evictAfter setting strategy
function setEvictDeadline(task: LocalAgentTaskState): number | undefined {
  if (task.isBackgrounded) {
    // background task: evictable after 1 hour
    return Date.now() + 60 * 60 * 1000;
  }
  if (task.retrieved) {
    // read foreground task: evict after 5 minutes
    return Date.now() + 5 * 60 * 1000;
  }
  return undefined;  // active tasks not evicted
}
```

9.4 Progress Tracking Algorithm

```
// LocalAgentTask.tsx: updateProgressFromMessage()

function updateProgressFromMessage(
  tracker: ProgressTracker,
  message: Message,
  resolveActivityDescription: ActivityDescriptionResolver,
  tools?: Tools,
): void {
  if (message.type !== 'assistant') return;

  // Step 1: update token count
  tracker.latestInputTokens = message.usage?.input_tokens
    ?? tracker.latestInputTokens;
  tracker.cumulativeOutputTokens +=
    message.usage?.output_tokens ?? 0;
```

```
  // Step 2: extract tool_use blocks
  const toolUses = extractToolUseBlocks(message);
  for (const toolUse of toolUses) {
    tracker.toolUseCount++;

    // Step 3: classify activity intensity
    const isSearch = tools?.isSearchTool(toolUse.name);
    const isRead = tools?.isReadTool(toolUse.name);

    // Step 4: build activity description
    const activity: ToolActivity = {
      toolName: toolUse.name,
      input: toolUse.input,
      activityDescription: resolveActivityDescription(message, toolUse.name),
      isSearch: isSearch ?? false,
      isRead: isRead ?? false,
    };

    tracker.recentActivities.push(activity);

    // Step 5: limit queue length (keep last 5)
    if (tracker.recentActivities.length > 5) {
      tracker.recentActivities.shift();
    }
  }
}

// getProgressUpdate: generate progress snapshot
function getProgressUpdate(tracker: ProgressTracker): AgentProgress {
  const recentNonSearch = tracker.recentActivities
    .filter(a => !a.isSearch)
    .slice(-3);

  // generate summary text
  const summary = recentNonSearch.length > 0
    ? `Last: ${recentNonSearch.map(a =>
        `${a.toolName}(${truncate(JSON.stringify(a.input), 50)})`
      ).join(' → ')}`
    : undefined;

  return {
    toolUseCount: tracker.toolUseCount,
    tokenCount: tracker.latestInputTokens + tracker.cumulativeOutputTokens,
    lastActivity: tracker.recentActivities.at(-1),
    recentActivities: tracker.recentActivities,
    summary,
  };
}
```

Chapter 10: Isolation Mechanisms

10.1 Worktree Isolation

```
// workflow
// 1. AgentTool.call() detects isolation: 'worktree'
// 2. create git worktree
// 3. fork subagent runs in worktree
// 4. notify parent session on completion

// create worktree
const worktreePath = path.join(
  repoRoot,
  '.claude',
  'worktrees',
  `agent-${agentId}`
);

const worktreeBranch = `claude/agent-${agentId}`;

// git worktree add <path> <branch> -b <new-branch>
await execGit(['worktree', 'add', '-B', worktreeBranch, worktreePath]);

// clone file read cache (preserve parent's cache hit info)
const clonedReadCache = cloneFileStateCache(
  toolUseContext.readFileState,
  {
    fromPath: repoRoot,
    toPath: worktreePath,
  }
);

// run agent in worktree
const worktreeToolUseContext = {
  ...toolUseContext,
  cwd: worktreePath,
  readFileState: clonedReadCache,
};

// post-completion handling
async function handleWorktreeCompletion(worktreePath: string, hasChanges: boolean) {
  if (!hasChanges) {
    // no changes: auto-cleanup worktree
    await execGit(['worktree', 'remove', worktreePath, '--force']);
    return undefined;
  }

  // has changes: keep worktree, notify user of path
  return {
    worktreePath,
    worktreeBranch,
    message: `Agent completed. Changes in: ${worktreePath}`,
  };
}
```

10.2 Remote Isolation (CCR only)

```
// isolation: 'remote' only available in ant internal builds
// via CCR (Claude Code Remote) protocol, schedules agent execution on remote machine

interface RemoteAgentParams {
  prompt: string;
  agentType: string;
  model?: string;
  remoteHost: string;
  credentials: AgentCredentials;
  // ... other params
}

// flow: remote agent runs on remote machine
// result asynchronously transmitted back to local via CCR protocol
// local receives <task-notification> format same as local async agent
```

Chapter 11: Coordinator Multi-Agent Orchestration

> **Note**
>
> See "Agent Team" documentation for details

11.1 Coordinator Mode Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                   Coordinator Mode Architecture                      │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   ┌──────────────────────────────────────────────────────────┐       │
│   │              Coordinator Agent                           │       │
│   │  Role: Orchestrator (doesn't execute directly)           │       │
│   │  Tools: Agent, SendMessage, TaskStop, Read, Bash, ... │   │
│   │  System Prompt: coordinatorMode.ts                       │       │
│   └──────────────────────────┬───────────────────────────────┘       │
│                              │                                        │
│       spawn worker           │          spawn worker                  │
│           ┌──────────────────┴──────────────────┐                    │
│           ▼                                     ▼                    │
│   ┌──────────────────┐              ┌──────────────────┐             │
│   │  Worker 1        │              │  Worker 2        │        │
│   │  type: "worker"  │              │  type: "worker"  │        │
│   │  Tools: restricted│             │  Tools: restricted│       │
│   │  Agent Teams: ✓  │              │  Agent Teams: ✓  │        │
│   └──────────┬───────┘              └──────────┬───────┘        │
│              │  task-notification              │                  │
│              └─────────────────────────────────┘                  │
│                              │                                     │
│                              ▼                                     │
│              Coordinator synthesizes results                      │
│              and responds to user                                 │
```

```
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```

11.2 Coordinator System Prompt Key Sections

```
// coordinatorMode.ts: getCoordinatorSystemPrompt()

// core role definition
const coordinatorPrompt = [
  `You are a task orchestration coordinator.`,
  `Your role is to PLAN, ASSIGN, and SYNTHESIZE — not to execute directly.`,

  // worker spawn decision matrix
  `USE SPAWN WHEN:`,
  `  - Task requires different tool sets (search vs edit)`,
  `  - Independent subtasks that can run in parallel`,
  `  - Specialized expertise (frontend, backend, infra)`,

  `USE CONTINUE WHEN:`,
  `  - Next step logically depends on current step's result`,
  `  - Context overlap > 60% with current task`,
  `  - Subtask is a natural continuation`,

  // worker prompt quality guide
  `SPAWNING BEST PRACTICES:`,
  `  - Write self-contained prompts: include all context the worker needs`,
  `  - Be specific about expected output format`,
  `  - Include constraints and edge cases`,
  `  - Keep prompts under 500 words for best results`,
  `  - Synthesize worker results: don't just concatenate`,

  // notification format explanation
  `HOW WORKER RESULTS ARRIVE:`,
  `  Worker results come as <task-notification> messages.`,
  `  These are user-role messages — read them to get worker output.`,
].join('\n');
```

11.3 Worker Tool Context

```
// coordinatorMode.ts: getCoordinatorUserContext()
// build restricted tool context for worker agent

function getCoordinatorUserContext(
  mcpClients: McpClients,
  scratchpadDir?: string,
): Record<string, string> {
  const tools: string[] = [
    // core tools
    'Read', 'Glob', 'Grep', 'WebSearch', 'WebFetch',
    'Write', 'Edit', 'Bash', 'NotebookEdit',
    // task tools
    'TodoWrite', 'TodoGet', 'TodoList', 'TodoUpdate',
```

```
    // team communication
    'SendMessage',
    // search enhancement
    'ToolSearch',
  ];

  // if scratchpad exists (cross-worker persistent directory), include its path
  if (scratchpadDir) {
    tools.push(
      `NOTE: Scratchpad at ${scratchpadDir} — write intermediate findings there.`
    );
  }

  return {
    available_tools: tools.join(', '),
    mcp_servers: Object.keys(mcpClients ?? {}).join(', ') || 'none',
  };
}
```

Chapter 12: End-to-End Data Flow

```
User Terminal
    │
    │ "Analyze the auth module and fix the session bugs"
    │
    ▼
┌──────────────────────────────────────────────────────────────────────┐
│                      Main Session                                    │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  Claude (Main)                                               │   │
│  │  thinking: This is complex. Let me spawn an Explore agent... │   │
│  │                                                              │   │
│  │  Tools:                                                      │   │
│  │  ┌──────────────────────────────────────────────────────┐   │   │
│  │  │  AgentTool.call({                                    │   │   │
│  │  │    prompt: "Find all auth/session related files",    │   │   │
│  │  │    subagent_type: "Explore",                         │   │   │
│  │  │  })                                                  │   │   │
│  │  └──────────────────────────────────────────────────────┘   │   │
│  └──────────────────────────────────────────────────────────────┘   │
└──────────────────────────────┬──────────────────────────────────────┘
                               │
          ┌────────────────────┼──────────────────────────────┐
          │ fork path          │ normal path                  │
          ▼                    ▼                              ▼
┌──────────────────┐  ┌──────────────────┐        ┌──────────────────┐
│  Fork Subagent   │  │  Explore Agent   │        │  [Other Agent]   │
│  (isolated       │  │  (sync one-shot) │        │  (async          │
│   worktree)      │  │                  │        │   background)    │
│                  │  │  useExactTools ✓ │        │                  │
```

```
│  Inherits:       │  │  forkCtx ✓        │        │ registerAsync    │
│  - system prompt │  │                   │        │ Agent()          │
│  - tool defs     │  │  for await msg   │        │                  │
│  - file cache    │  │  of query()      │        │ runAsync         │
│                  │  │  → yield msg     │        │ AgentLifecycle() │
│  buildForked     │  │                   │        │  ↓               │
│  Messages()      │  │  return result   │        │  enqueue         │
│  ↓               │  │                   │        │  Notification() │
│  query()         │  └──────────┬────────┘        └──────────┬───────┘
│  (cache hit ✓)   │           │                              │
│                  │           ▼                              ▼
│                  │  ┌──────────────────────────────────────────┐
│                  │  │         Results Aggregation              │
│                  │  │                                          │
│                  │  │  AgentToolResult {                       │
│                  │  │    agentId: "agent-abc",                 │
│                  │  │    content: "Found 12 files: ...",       │
│                  │  │    totalToolUseCount: 7,                  │
│                  │  │    totalTokens: 4820,                     │
│                  │  │    totalDurationMs: 8430,                 │
│                  │  │    usage: { input_tokens: 1204, ... }    │
│                  │  │  }                                        │
│                  │  └──────────────────────────────────────────┘
│                  │           │
└──────────────────┴───────────┴──────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────────┐
│                     Main Session (continued)                      │
│                                                                  │
│  Agent result received. Now fixing the bugs...                   │
│                                                                  │
│  Another AgentTool.call() → spawn general-purpose agent          │
│  (with verification) to make the fixes                           │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │  <task-notification> (from background agents)            │   │
│  │  When async agents complete, their results appear here   │   │
│  │  as user-role messages in the main conversation          │   │
│  └──────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────┘
```

Chapter 13: Key Algorithm Pseudocode Implementations

13.1 Tool Filtering Algorithm

```
ALGORITHM: filterToolsForAgent
INPUT:
  - allTools: Tool[]          all available tools
  - agentDef: AgentDefinition  agent definition
  - isAsync: boolean           whether async execution
  - permissionMode: PermissionMode  permission mode
```

```
OUTPUT: filteredTools: Tool[]

BEGIN
  filtered ← allTools

  # Layer 1: Global Blocklist
  filtered ← filtered EXCEPT ALL_AGENT_DISALLOWED_TOOLS
  IF USER_TYPE ≠ 'ant' THEN
    filtered ← filtered EXCEPT 'Agent'
  END IF

  # Layer 2: Async Mandatory Allowlist
  IF isAsync = TRUE THEN
    filtered ← filtered INTERSECT ASYNC_AGENT_ALLOWED_TOOLS
  END IF

  # Layer 3: Custom Agent Extra Blocks
  IF agentDef.source ∈ {userSettings, projectSettings} THEN
    filtered ← filtered EXCEPT CUSTOM_AGENT_DISALLOWED_TOOLS
  END IF

  # Layer 4: Agent Allowlist
  IF agentDef.tools EXISTS THEN
    IF agentDef.tools ≠ ['*'] THEN
      filtered ← filtered INTERSECT agentDef.tools
    END IF
  END IF

  # Layer 5: Agent Blocklist (highest priority)
  IF agentDef.disallowedTools EXISTS THEN
    filtered ← filtered EXCEPT agentDef.disallowedTools
  END IF

  # Layer 6: Permission Mode Adjustment
  SWITCH permissionMode:
    CASE 'bypassPermissions':
      filtered ← filtered.map(t → t.removePermissionWrapper())
    CASE 'acceptEdits':
      filtered ← filtered.map(t →
        IF t.name ∈ {Edit, Write} THEN t.withAutoApprove() ELSE t
      )
    CASE 'plan':
      filtered ← filtered.map(t → t.wrapWithPlanApproval())
  END SWITCH

  RETURN filtered
END
```

13.2 Fork Message Building Algorithm

```
ALGORITHM: buildForkedMessages
INPUT:
```

```
  - directive: string           child Agent's task directive
  - assistantMessage: Message  parent Agent's last assistant message

OUTPUT: forkedMessages: Message[]

BEGIN
  # Step 1: deep copy (avoid polluting parent's REPL state)
  cloned ← DEEP_CLONE(assistantMessage)

  # Step 2: extract tool_use blocks
  toolUses ← cloned.content WHERE block.type = 'tool_use'

  # Step 3: create byte-identical placeholder results for each tool_use
  # Key: FORK_PLACEHOLDER_RESULT is a constant string
  # API's tool_result serialization is identical → cache hit
  toolResultBlocks ← []
  FOR EACH toolUse IN toolUses DO
    APPEND toolResultBlocks ← {
      type: 'tool_result',
      tool_use_id: toolUse.id,
      content: [{
        type: 'text',
        text: FORK_PLACEHOLDER_RESULT,
        is_error: false,
      }]
    }
  END FOR

  # Step 4: build user directive message (new, not part of cache)
  directiveMsg ← {
    type: 'user',
    content: [{
      type: 'text',
      text: buildChildMessage(directive),  # 10 FORK_BOILERPLATE_TAG rules
    }]
  }

  # Step 5: assemble message sequence
  RETURN [cloned] + toolResultBlocks + [directiveMsg]
END

FUNCTION buildChildMessage(directive)
  RETURN "<FORK_BOILERPLATE_TAG>\n" +
    "Task: " + directive + "\n" +
    "Rules: " +
    "1. Continue from previous agent's last action\n" +
    "2. Complete the task autonomously\n" +
    "3. Report completion with concise summary\n" +
    "4. For multi-step: execute all remaining steps\n" +
    "5. One-shot: run to completion without asking\n" +
    "6. Tool restrictions override your default capabilities\n" +
    "7. If blocked: attempt alternatives\n" +
    "8. Do not explain reasoning unless asked\n" +
```

```
    "9. Result goes to parent session\n" +
    "10. You control execution — parent is waiting\n" +
    "</FORK_BOILERPLATE_TAG>"
END FUNCTION
```

13.3 Progress Tracking Algorithm

```
ALGORITHM: updateProgressFromMessage
INPUT:
  - tracker: ProgressTracker     runtime tracker
  - message: Message              message from query()
  - resolveActivityDescription:  description resolver function
  - tools: Tool[]                 available tool set

BEGIN
  IF message.type ≠ 'assistant' THEN
    RETURN  # only process assistant messages
  END IF

  # Step 1: update token count
  IF message.usage EXISTS THEN
    tracker.latestInputTokens ← message.usage.input_tokens
    tracker.cumulativeOutputTokens ←
      tracker.cumulativeOutputTokens + message.usage.output_tokens
  END IF

  # Step 2: process tool_use blocks
  toolUses ← EXTRACT_TOOL_USE_BLOCKS(message)
  FOR EACH toolUse IN toolUses DO
    tracker.toolUseCount ← tracker.toolUseCount + 1

    # Step 3: classify tool intensity (search=low intensity, not counted as "active")
    isSearch ← tools?.isSearchTool(toolUse.name) ?? FALSE
    isRead ← tools?.isReadTool(toolUse.name) ?? FALSE

    # Step 4: build activity record
    activity ← {
      toolName: toolUse.name,
      input: toolUse.input,
      activityDescription: resolveActivityDescription(
        message, toolUse.name
      ),
      isSearch,
      isRead,
    }

    # Step 5: append to recent activities queue (max 5)
    APPEND tracker.recentActivities ← activity
    IF LENGTH(tracker.recentActivities) > 5 THEN
      SHIFT tracker.recentActivities  # remove oldest
    END IF
  END FOR
END
```

```
ALGORITHM: getProgressUpdate
INPUT: tracker: ProgressTracker

OUTPUT: AgentProgress

BEGIN
  # generate summary: take recent non-search activities
  recentNonSearch ← FILTER(
    tracker.recentActivities,
    a → NOT a.isSearch
  )[-3:]  # last 3

  summary ← ""
  IF LENGTH(recentNonSearch) > 0 THEN
    summary ← "Last: " + JOIN(
      recentNonSearch.map(a →
        a.toolName + "(" + TRUNCATE(JSON.stringify(a.input), 50) + ")"
      ),
      " → "
    )
  END IF

  RETURN {
    toolUseCount: tracker.toolUseCount,
    tokenCount: tracker.latestInputTokens + tracker.cumulativeOutputTokens,
    lastActivity: LAST(tracker.recentActivities),
    recentActivities: tracker.recentActivities,
    summary: summary IF summary ≠ "" ELSE undefined,
  }
END
```

Chapter 14: Design Patterns and Architectural Philosophy

14.1 Factory Pattern: Agent Definition Registry

```typescript
// builtInAgents.ts
// unified agent acquisition entry, supports feature flags and dynamic registration

class AgentRegistry {
  private agents: Map<string, AgentDefinition> = new Map();

  register(agent: AgentDefinition): void {
    // same type override (built-in takes precedence over custom)
    if (agent.source === 'built-in' || !this.agents.has(agent.agentType)) {
      this.agents.set(agent.agentType, agent);
    }
  }

  get(type: string): AgentDefinition | undefined {
    return this.agents.get(type);
  }
```

```
  getAll(): AgentDefinition[] {
    return [...this.agents.values()];
  }
}

// usage
function resolveAgent(agentType: string): AgentDefinition {
  return (
    builtInAgents.get(agentType) ??
    customAgents.get(agentType) ??
    generalPurposeAgent  // default fallback
  );
}
```

14.2 Strategy Pattern: Tool Filtering Strategy

```
// different agent types → different filtering strategies
const filterStrategies = {
  async:      (tools, agentDef) => filterToolsAsync(tools, agentDef),
  builtin:    (tools, agentDef) => filterToolsBuiltin(tools, agentDef),
  custom:     (tools, agentDef) => filterToolsCustom(tools, agentDef),
  coordinator:(tools, agentDef) => filterToolsCoordinator(tools, agentDef),
};

// in runAgent.ts, dispatch by type
const strategy = filterStrategies[agentCategory] ?? filterStrategies.builtin;
const filteredTools = strategy(allTools, agentDef);
```

14.3 Iterator Pattern: Message Stream

```
// runAgent.ts: core iteration pattern
// runAgent() is AsyncGenerator<Message>, consumes query() message stream

async function* runAgent(params): AsyncGenerator<Message> {
  // pre-setup...
  try {
    for await (const message of query(queryOptions)) {
      // each message is yielded in real-time to caller (AgentTool.call)
      yield message;
    }
  } finally {
    // whether normal or exception, finally ensures cleanup
    await cleanup();
  }
}

// consumption in AgentTool.tsx
for await (const message of runAgent(params)) {
  handleMessage(message);
}
```

14.4 Template Method: Agent Execution Skeleton

```
// runAgent.ts: define skeleton, subclasses/config can customize each step
class AgentExecutor {
  async* execute(params: RunAgentParams): AsyncGenerator<Message> {
    await this.setup(params);           // fixed: initialization
    const options = await this.buildQueryOptions(params); // overridable
    try {
      for await (const message of query(options)) {
        yield message;
      }
    } finally {
      await this.cleanup(params);       // fixed: cleanup
    }
  }

  async setup(params) { /* initialization */ }
  async buildQueryOptions(params) { /* build options */ }
  async cleanup(params) { /* cleanup */ }
}
```

14.5 Decorator Pattern: Permission Wrapping

```
// tool calls are wrapped by permission decorators, dynamically select behavior by
permission mode

function wrapToolWithPermission(
  tool: Tool,
  permissionMode: PermissionMode,
  canUseTool: CanUseToolFn,
): Tool {
  switch (permissionMode) {
    case 'bypassPermissions':
      // zero checks, execute directly
      return tool;

    case 'acceptEdits':
      // Edit/Write auto-approved, others normal approval
      if (tool.name === 'Edit' || tool.name === 'Write') {
        return tool.withAutoApprove();
      }
      return tool.withPermissionCheck(canUseTool);

    case 'plan':
      // all tools require plan first
      return tool.withPlanApproval();

    case 'bubble':
    default:
      // normal permission check, bubble up
      return tool.withPermissionCheck(canUseTool);
```

```
  }
}
```

14.6 Publish-Subscribe Pattern: Progress Notification

```
// LocalAgentTask.tsx: ProgressTracker implements pub-sub
class ProgressTracker {
  private listeners: Array<(progress: AgentProgress) => void> = [];
  private tracker: ProgressTracker;  // internal state

  // subscribe
  subscribe(listener: (progress: AgentProgress) => void): () => void {
    this.listeners.push(listener);
    // return unsubscribe function
    return () => {
      this.listeners = this.listeners.filter(l => l !== listener);
    };
  }

  // publish
  private notify(): void {
    const progress = getProgressUpdate(this.tracker);
    this.listeners.forEach(listener => listener(progress));
  }

  // auto-publish on update
  update(message: Message, ...args): void {
    updateProgressFromMessage(this.tracker, message, ...args);
    this.notify();  // auto-notify all subscribers
  }
}
```

14.7 Command Pattern: Task Lifecycle

```
// LocalAgentTask.tsx: each task operation is an immutable state transition
type TaskCommand =
  | { type: 'start'; payload: RunAgentParams }
  | { type: 'background'; payload: { taskId } }
  | { type: 'complete'; payload: { result: AgentToolResult } }
  | { type: 'fail'; payload: { error: string } }
  | { type: 'kill' }
  | { type: 'discard' };

// state machine (using Immer or similar immutable update)
function reduceTaskState(
  state: LocalAgentTaskState,
  command: TaskCommand
): LocalAgentTaskState {
  switch (command.type) {
    case 'background':
      return { ...state, isBackgrounded: true };
    case 'complete':
```

```
      return { ...state, status: 'completed', result: command.payload.result };
    case 'fail':
      return { ...state, status: 'failed', error: command.payload.error };
    case 'kill':
      return { ...state, status: 'killed' };
    default:
      return state;
  }
}
```

Chapter 15: Security and Boundary Control

15.1 Security Boundary Overview

```
┌──────────────────────────────────────────────────────────────────────┐
│                        Security Boundary                             │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   ┌──────────────────────────────────────────────────────────────┐   │
│   │                    User Session                              │   │
│   │                                                              │   │
│   │   Tool Call Boundary (AgentTool.call)                        │   │
│   │   ├── canUseTool('Agent') — permission verification          │   │
│   │   ├── tool filtering — remove dangerous tools                │   │
│   │   ├── MCP server verification — requiredMcpServers           │   │
│   │   ├── recursion depth guard — prevent fork nesting           │   │
│   │   └── Abort propagation chain — parent abort → all kids      │   │
│   │                                                              │   │
│   │   ┌──────────────────────────────────────────────────────┐   │   │
│   │   │          Sub-Agent Sandbox                           │   │   │
│   │   │                                                      │   │   │
│   │   │   • independent AbortController (linked to parent)   │   │   │
│   │   │   • Worktree isolation (file changes don't           │   │   │
│   │   │     pollute parent)                                  │   │   │
│   │   │   • cloned file read cache (doesn't leak parent's    │   │   │
│   │   │     temp files)                                      │   │   │
│   │   │   • independent MCP clients (don't share session-    │   │   │
│   │   │     level MCP)                                       │   │   │
│   │   │   • independent Frontmatter Hooks (don't inherit     │   │   │
│   │   │     parent's hooks)                                  │   │   │
│   │   │                                                      │   │   │
│   │   │   ┌──────────────────────────────────────────┐       │   │   │
│   │   │   │  Tool Execution Sandboxing               │       │   │   │
│   │   │   │                                          │       │   │   │
│   │   │   │  Edit/Write: permissionMode control      │       │   │   │
│   │   │   │  Bash: shell permission mode control     │       │   │   │
│   │   │   │  WebSearch/Fetch: isolated HTTP client   │       │   │   │
│   │   │   │  MCP: independent connection, don't      │       │   │   │
│   │   │   │       inherit parent session's MCP state │       │   │   │
│   │   │   │                                          │       │   │   │
│   │   │   └──────────────────────────────────────────┘       │   │   │
│   │   └──────────────────────────────────────────────────────┘   │   │
```

```
│   └──────────────────────────────────────────────────────────────┘   │
│                                                                        │
│  Post-Execution Security Review:                                       │
│  ├── classifyHandoffIfNeeded() — check agent's tool usage             │
│  ├── sensitive file access review — did agent access                   │
│  │   inappropriate files                                               │
│  └── abnormal tool call review — did agent bypass                      │
│       tool restrictions                                                │
│                                                                        │
└────────────────────────────────────────────────────────────────────────┘
```

15.2 Handoff Security Classification

```
// agentToolUtils.ts: classifyHandoffIfNeeded()
// executed after agent completes, used to review child agent's behavior

async function classifyHandoffIfNeeded(params: {
  agentId: string;
  result: AgentToolResult;
  toolUseContext: ToolUseContext;
}): Promise<string | null> {
  // feature flag: off by default
  if (!isHandoffClassifierEnabled()) {
    return null;
  }

  // collect all tool calls made by agent
  const allToolCalls = extractAllToolCalls(params.result.agentMessages);

  // review dimensions
  const concerns: string[] = [];

  // 1. check if sensitive paths were accessed
  const sensitivePaths = [
    process.env.HOME + '/.ssh/',
    process.env.HOME + '/.aws/',
    '/etc/secrets/',
  ];
  for (const call of allToolCalls) {
    if (call.tool === 'Read' || call.tool === 'Glob') {
      for (const path of sensitivePaths) {
        if (call.input.path?.startsWith(path)) {
          concerns.push(`Sensitive path access: ${path}`);
        }
      }
    }
  }

  // 2. check for unusually high write operations (possible anomaly)
  const writeCount = allToolCalls.filter(
    c => c.tool === 'Edit' || c.tool === 'Write'
  ).length;
  if (writeCount > 50) {
```

```
    concerns.push(`Unusually high write count: ${writeCount}`);
  }

  // 3. check for unexpected tool usage
  const allowedTools = ASYNC_AGENT_ALLOWED_TOOLS;
  for (const call of allToolCalls) {
    if (!allowedTools.has(call.tool)) {
      concerns.push(`Unexpected tool: ${call.tool}`);
    }
  }

  // return classification result
  if (concerns.length === 0) {
    return 'CLEAR';
  }
  return concerns.join('; ');
}
```

15.3 Abort Propagation Chain

```
// forkedAgent.ts: createSubagentContext()
// subagent's AbortController is linked to parent's AbortController

function createChildAbortController(
  parentController: AbortController | undefined,
  agentId: string,
): AbortController {
  const child = new AbortController();

  if (parentController) {
    // parent abort → automatically triggers child's abort
    parentController.signal.addEventListener('abort', () => {
      child.abort(`Parent session cancelled (agent: ${agentId})`);
    }, { once: true });
  }

  return child;
}

// usage scenarios
// 1. user presses ESC → main session aborts → propagates to all subagents
// 2. parent agent completes → aborts still-running subagents
// 3. tool timeout → abort current agent
```

15.4 Path Traversal Protection

```
// agentMemory.ts: isAgentMemoryPath()
// ensure agent cannot access files outside memory directory via ../../../

function isAgentMemoryPath(absolutePath: string): boolean {
  const normalized = path.normalize(absolutePath);
```

```
  const memoryBase = path.join(os.homedir(), '.claude', 'agent-memory');
  const projectMemoryBase = path.join(process.cwd(), '.claude', 'agent-memory');
  const remoteMemoryBase = process.env.CLAUDE_CODE_REMOTE_MEMORY_DIR ?? '';

  return (
    normalized.startsWith(memoryBase) ||
    normalized.startsWith(projectMemoryBase) ||
    (remoteMemoryBase !== '' && normalized.startsWith(remoteMemoryBase))
  );
}

// usage: in all file reads in agentMemory.ts
function loadAgentMemoryPrompt(agentType: string, scope: AgentMemoryScope) {
  const memoryPath = getAgentMemoryEntrypoint(agentType, scope);

  // security check: prevent bypass via ../../../etc/passwd
  if (!isAgentMemoryPath(memoryPath)) {
    throw new Error(`Security: attempted memory path traversal: ${memoryPath}`);
  }

  return readFileSync(memoryPath, 'utf-8');
}
```

Appendix: Constants Quick Reference

```
// ==================== Tool Names ====================
AGENT_TOOL_NAME           = 'Agent'
LEGACY_AGENT_TOOL_NAME    = 'Task'

// ==================== Agent Types ====================
FORK_SUBAGENT_TYPE        = 'fork'          // Fork subagent virtual type
WORKER_AGENT_TYPE         = 'worker'        // Coordinator worker type

// ==================== One-shot Agents ====================
// these agents don't need SendMessage trailer, return directly
ONE_SHOT_BUILTIN_AGENT_TYPES = new Set(['Explore', 'Plan'])

// ==================== Fork Constants ====================
FORK_BOILERPLATE_TAG      = 'FORK_BOILERPLATE'
FORK_PLACEHOLDER_RESULT   = 'Fork started — processing in background'

// ==================== Time Thresholds ====================
PROGRESS_THRESHOLD_MS     = 2000            // delay before showing background hint (ms)
AUTO_BACKGROUND_MS        = 120_000         // auto-background timeout (ms, default)
DEFAULT_AGENT_TIMEOUT_MS  = 600_000         // default agent timeout (10 minutes)

// ==================== Memory Management ====================
MAX_RECENT_ACTIVITIES     = 5               // progress tracking recent activity count
MAX_MEMORY_INDEX_LINES    = 200             // MEMORY.md index max lines
DEFAULT_EVICT_AFTER_MS    = 5 * 60 * 1000   // foreground task evictable after 5 minutes
BG_EVICT_AFTER_MS         = 60 * 60 * 1000  // background task evictable after 1 hour
```

```
// ==================== Token Limits ====================
DEFAULT_MAX_OUTPUT_TOKENS = 8192            // default max output tokens
MAX_OUTPUT_TOKENS_HAIKU  = 4096            // Haiku model limit
```